

Finetuning Large Language Models

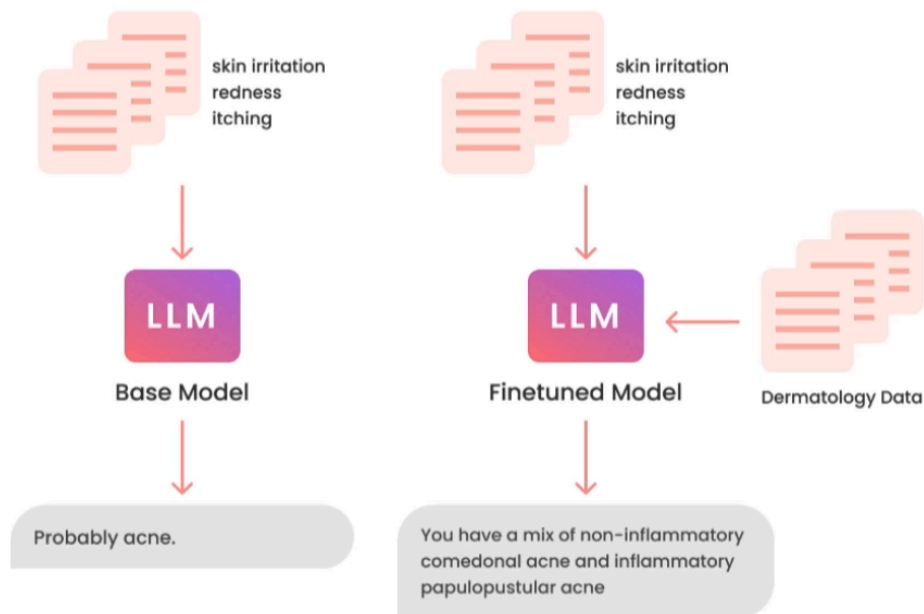
🕒 Created	July 28, 2025 4:53 PM
👤 OfferedBy	Deeplearning.ai
🌐 Platform	Deeplearning.ai
👥 Instructors	Sharon Zhou
🛠 Tools	Lamini
📂 Topic	Finetuning

1. What is Fine tuning
2. When to finetune
3. How it fits into training and differs from prompt engineering and RAG
4. How to use these techniques alongside fine-tuning
5. GPT-3 to Chat GPT via Instruct Fine tuning

Introduction

What is Finetuning

taking in general purpose models and specializing them. Analogy:" Turning primary care physician into a specialist. For example, skin irritation and redness symptoms to a PCP would say acne, but a model trained on dermatology data would give a much more clearer and specific diagnosis



What does finetuning do for the model

1. Steers the models for more consistent output
2. Reduces hallucinations
3. Customizes the model to a specific use-case
4. Process similar to model's prior training

Prompt Engineering vs Finetuning

	prompting	Finetuning
Pros	No data to get started Smaller upfront cost no technical knowledge Connect data through <u>Retrieval augmented generation (RAG)</u>	Nearly unlimited data fits Learn new information Correct incorrect information Less cost afterwards if smaller model Use RAG too
Cons	Much less data fits Forgets data Hallucinations RAG misses or gets incorrect data	More high quality data Upfront compute cost Need some technical knowledge especially data.
Usage	Generic, Side projects, prototypes	Domain-specific, enterprise, production usage, privacy

Benefits of finetuning your own LLM

Performance Benefits

- Stop hallucinations
- Increase consistency and reliability
- Reduce unwanted info - can be moderated

Privacy benefits

- On-prem or VPC
- Prevent leakage
- No data breaches

Cost

- Lower cost per request - finetuning a smaller LLM
- Increase transparency
- Greater control

Reliability

- Control uptime
- Lower latency
- Moderation - provides guardrails and custom responses

Tools

-  [PyTorch](#)
-  [Open Source Models with HuggingFace](#)
- Llama library (Lamini)

Code

```
import os
import lamini

lamini.api_url = os.getenv("POWERML__PRODUCTION__URL")
lamini.api_key = os.getenv("POWERML__PRODUCTION__KEY")

from llama import BasicModelRunner
```

```

non_finetuned = BasicModelRunner("meta-llama/Llama-2-7b-hf")
non_finetuned_output = non_finetuned("Tell me how to train my dog to sit")
print(non_finetuned_output)

finetuned_model = BasicModelRunner("meta-llama/Llama-2-7b-chat-hf")
finetuned_output = finetuned_model("Tell me how to train my dog to sit")
print(finetuned_output)

# Avoids Autocomplete of the model by wrapping the command in INST (instruction tags)
print(finetuned_model("[INST]Tell me how to train my dog to sit[/INST]"))

```

Finetuning data: compare to pretraining and basic preparation

Pretraining

- Model at the start
 - zero knowledge about the world
 - can't form English words
- Next word prediction
- Giant corpus of text
- Often scraped from the internet: "unlabelled"
 - Often not publicized how to pretrain
 - Open-source pretraining data - "The Pile" - 22 diverse datasets scraped from the internet
 - Expensive and time consuming to train
- Self-supervised learning
- After training

- Learns language
- Learns knowledge

Finetuning after pre-training

- Not a chatbot interface at first. Finetune.
- Finetuning usually refers to training further
 - Can also be self-supervised unlabeled data
 - Can be labelled data you curated
 - Much less data needed
- Finetuning for generative tasks is not well-defined
 - Updates entire model, not just part of it
 - Same training objective: next token prediction - just changing the data

What is finetuning doing?

- Behavior change
 - Learn to respond more consistently
 - Learning to focus e.g. moderation
 - Teasing out capability - better at conversation
- Gain knowledge
 - Increasing knowledge of new specific concepts
 - Correcting old incorrect information
- Both

Tasks to finetune

- Just text-in, text-out
 - Extraction: text in, less text out
 - Reading
 - Keywords, topics, routing, agents (planning, reasoning, self-critic, tool use), etc
 - Expansion: Text in, more text out

- Task clarity is the key indicator of success
- Clarity means knowing what's bad vs good vs better

First time finetuning

1. Identify tasks by prompt engineering a LLM
2. Find tasks that you see LLM doing OK at
3. Pick one task
4. Get 1000 inputs and outputs for the task - Better than the OK output from LLM
5. Finetune a small LLM on this data

Lab 2: Finetuning data: compare to pretraining and basic preparation

02_Where_finnetuning_fits_in_lab_student.ipynb

```
import jsonlines
import itertools
import pandas as pd
from pprint import pprint

import datasets # from huggingface
from datasets import load_dataset
```

Look at pretraining data set

Sorry, "The Pile" dataset is currently relocating to a new home and so we can't show you the same example that is in the video. Here is another dataset, the "Common Crawl" dataset.

```
#pretrained_dataset = load_dataset("EleutherAI/pile", split="train", streaming=True)
pretrained_dataset = load_dataset("c4", "en", split="train", streaming=True)
```

```
n = 1
print("Pretrained dataset:")
top_n = itertools.islice(pretrained_dataset, n)
for i in top_n:
    print(i)
```

```
Pretrained dataset:
{'text': 'Beginners BBQ Class Taking Place in Missoula!\\nDo you want to get better at making delicious BBQ? You will have the opportunity, put this on your calendar now. Thursday, September 22nd join World Class BBQ Champion, Tony Balay from Lonestar Smoke Rangers. He will be teaching a beginner level class for everyone who wants to get better with their culinary skills.\\nHe will teach you everything you need to know to compete in a KCBS BBQ competition, including techniques, recipes, timelines, meat selection and trimming, plus smoker and fire information.\\nThe cost to be in the class is $35 per person, and for spectators it is free. Included in the cost will be either a t-shirt or apron and you will be tasting samples of each meat that is prepared.', 'timestamp': '2019-04-25T12:57:54Z', 'url': '<https://klyq.com/beginners-bbq-class-taking-place-in-missoula/>'}
```

Contrast with company finetuning dataset you will be using

```
#FAQ
filename = "lamini_docs.jsonl"
instruction_dataset_df = pd.read_json(filename, lines=True)
```

Various ways of formatting your data

Simple concatenation vs prompt template

```
examples = instruction_dataset_df.to_dict()
text = examples["question"][0] + examples["answer"][0]
```

```
"What are the different types of documents available in the repository (e.g., installation guide, API documentation, developer's guide)?Lamini has documentation on Getting
```

Started, Authentication, Question Answer Model, Python Library, Batching, Error Handling, Advanced topics, and class documentation on LLM Engine available at <<https://lamini-ai.github.io/>>.">

```
if "question" in examples and "answer" in examples:
    text = examples["question"][0] + examples["answer"][0]
elif "instruction" in examples and "response" in examples:
    text = examples["instruction"][0] + examples["response"][0]
elif "input" in examples and "output" in examples:
    text = examples["input"][0] + examples["output"][0]
else:
    text = examples["text"][0]
```

```
prompt_template_qa = """### Question:
{question}

### Answer:
{answer}"""
```

```
question = examples["question"][0]
answer = examples["answer"][0]

text_with_prompt_template = prompt_template_qa.format(question=question, answer=answer)
text_with_prompt_template
```

```
"""### Question:\nWhat are the different types of documents available in the repository
(e.g., installation guide, API documentation, developer's guide)?\n\n### Answer:\nL
amini has documentation on Getting Started, Authentication, Question Answer Model, Pyt
hon Library, Batching, Error Handling, Advanced topics, and class documentation on LLM
Engine available at <https://lamini-ai.github.io/>.">
```

```
prompt_template_q = """### Question:
{question}
```



```
### Answer:""
```

```
num_examples = len(examples["question"])
finetuning_dataset_text_only = []
finetuning_dataset_question_answer = []
for i in range(num_examples):
    question = examples["question"][i]
    answer = examples["answer"][i]

    text_with_prompt_template_qa = prompt_template_qa.format(
        question=question, answer=answer)
    finetuning_dataset_text_only.append({"text": text_with_prompt_template_qa})

    text_with_prompt_template_q = prompt_template_q.format(question=question)
    finetuning_dataset_question_answer.append({"question": text_with_prompt_template_q, "answer": answer})
```

```
pprint(finetuning_dataset_text_only[0])
```

```
{'text': '### Question:\\n'
        'What are the different types of documents available in the '
        'repository (e.g., installation guide, API documentation, developer's '
        'guide)?\\n'
        '\\n'
        '### Answer:\\n'
        'Lamini has documentation on Getting Started, Authentication, '
        'Question Answer Model, Python Library, Batching, Error Handling, '
        'Advanced topics, and class documentation on LLM Engine available at '
        '<https://lamini-ai.github.io/.>'}
```

```
pprint(finetuning_dataset_question_answer[0])
```

```
{'answer': 'Lamini has documentation on Getting Started, Authentication, '
           'Question Answer Model, Python Library, Batching, Error Handling, '
           'Advanced topics, and class documentation on LLM Engine available '}
```

```
'at <https://lamini-ai.github.io/.>',
'question': '### Question:\\n'
            'What are the different types of documents available in the '
            'repository (e.g., installation guide, API documentation, '
            '"developer's guide")?\\n"
            '\\n'
            '### Answer: '}]
```

Common ways of storing your data

- saving as `jsonl` files - one json object per line
- uploading to 🤗 [HuggingFace](#)

```
with jsonlines.open(f'lamini_docs_processed.jsonl', 'w') as
writer:
    writer.write_all(finetuning_dataset_question_answer)
```

```
finetuning_dataset_name = "lamini/lamini_docs"
finetuning_dataset = load_dataset(finetuning_dataset_name)
print(finetuning_dataset)
```

Instruction Fine-tuning

Subset of finetuning where it teaches the model to follow instructions and behave like a chatbot. This is the method that turned GPT-3 into ChatGPT and as a result massively increased AI adoption from thousands of researchers to millions of people.

Instruction following datasets

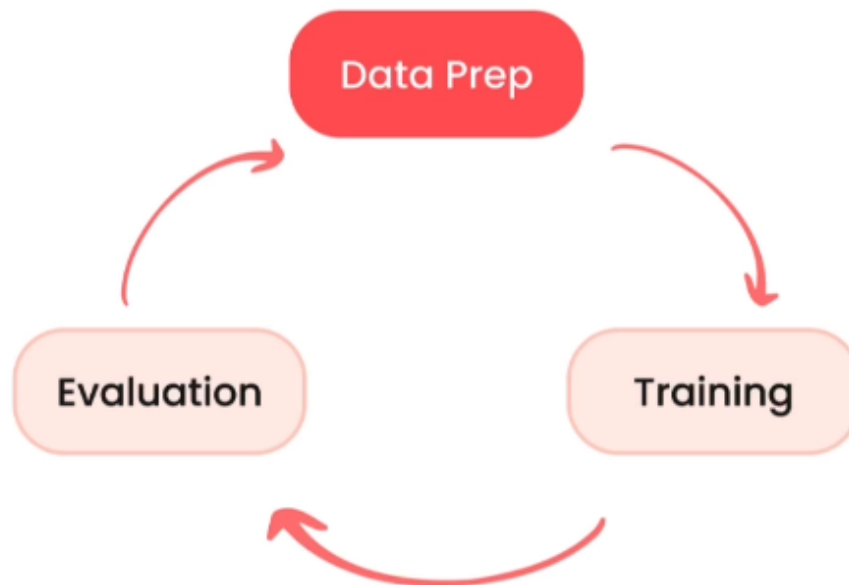
Can use existing datasets like FAQs, customer support conversations and slack messages which are instruction-response datasets.

If no data,

- then convert the data into question answer format using a prompt template. For example, converting a readme into a question answer pair.
- Can also use another LLM to do this for you. There is a technique called  [Alpaca](#) that uses ChatGPT to convert data into question answer pairs or can use a pipeline of different open source models to do this as well.

Finetuning steps

It is an iterative process



- Data preparation
- Training
- Evaluation

```
from datasets import load_dataset
instruction_tuned_dataset = load_dataset("tatsu-lab/alpaca", split="train", streaming=True) # streaming for large datasets
m = 5
top_m = list(itertools.islice(instruction_tuned_dataset, m))
top_m[0]
```

```
{'instruction': 'Give three tips for staying healthy.',
 'input': '',
 'output': '1.Eat a balanced diet and make sure to include plenty of fruits and vegetables. \n2. Exercise regularly to keep your body active and strong. \n3. Get enough sleep and maintain a consistent sleep schedule.'}
```

```
'text': 'Below is an instruction that describes a task. Write a response that appropriately completes the request.\n\n### Instruction:\nGive three tips for staying healthy.\n\n### Response:\n1. Eat a balanced diet and make sure to include plenty of fruits and vegetables. \n2. Exercise regularly to keep your body active and strong. \n3. Get enough sleep and maintain a consistent sleep schedule.'
```

Prompt templates

```
prompt_template_with_input = """Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.
```

```
### Instruction:
{instruction}
```

```
### Input:
{input}
```

```
### Response: """
```

```
prompt_template_without_input = """Below is an instruction that describes a task. Write a response that appropriately completes the request.
```

```
### Instruction:
{instruction}
```

```
### Response: """
```

```
processed_data = []
for j in top_m:
    if not j["input"]:
        processed_prompt = prompt_template_without_input.format(instruction=j["instruction"])
    else:
```

```

        processed_prompt = prompt_template_with_input.format(
            instruction=j["instruction"], input=j["input"])

        processed_data.append({"input": processed_prompt, "output": j["output"]})

```

Save dataset to `jsonl`

```

with jsonlines.open(f'alpaca_processed.jsonl', 'w') as writer:
    writer.write_all(processed_data)

```

[03_Instruction_tuning_lab_student.ipynb](#)

Data Preparation

What kind of data is required for finetuning

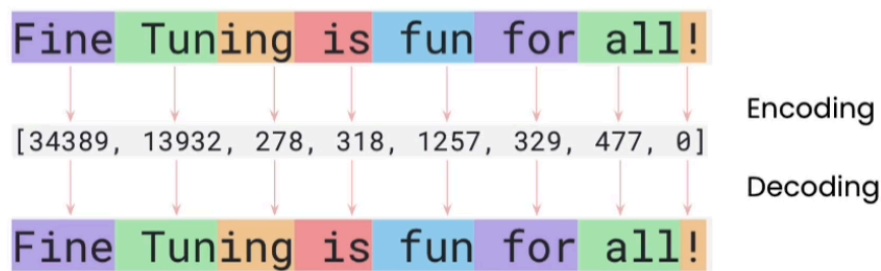
- Higher quality - Garbage in garbage out
- Diversity - low diversity, model tends to memorize
- Real - more effective for writing tasks
- More - more data helps less data (but not as important as the first 3)

Steps to data preparation

- Collect instruction response pairs
- Concatenate pairs, add prompt template if applicable
- Tokenize - Pad, Truncate
- Split into train and test

Tokenizing

Convert string into numerical representation



There are multiple popular tokenizers

- use the tokenizers associated with your model
- using wrong tokenizer can confuse the model, because it will expect different numbers

Lab: Data preparation

`AutoTokenizer` automatically finds the right tokenizer for the model based on the model specified.

Tokenization

```
import pandas as pd
import datasets
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained('EleutherAI/pythia-70m')
text = 'Hi!! How are you?'
encoded_text = tokenizer(text)['input_ids']
decoded_text = tokenizer.decode(encoded_text)
print("Decoded tokens back into text: ", decoded_text)
```

Tokenize Multiple texts

```
list_texts = ["Hi, how are you?", "I'm good", "Yes"]
encoded_texts = tokenizer(list_texts)
print("Encoded several texts: ", encoded_texts["input_ids"])
```

Padding

```
# pad_token returns nothing
tokenizer.pad_token = tokenizer.eos_token
```

```
encoded_texts_longest = tokenizer(list_texts, padding=True)
print("Using padding: ", encoded_texts_longest["input_ids"])
```

Truncation

```
# gets rid of everything on the right side of the text
encoded_texts_truncation = tokenizer(list_texts, max_length=3, truncation=True)

# can specify side left to retain right side information
tokenizer.truncation_side = "left"
encoded_texts_truncation_left = tokenizer(list_texts, max_length=3, truncation=True)
print("Using left-side truncation: ", encoded_texts_truncation_left["input_ids"])
encoded_texts_both = tokenizer(list_texts, max_length=3, truncation=True, padding=True)
print("Using both padding and truncation: ", encoded_texts_both["input_ids"])
```

[04_Data_preparation_lab_student.html](#)

[04_Data_preparation_lab_student.ipynb](#)

Training

- Add training data
- Predict
- Calculate loss
- Backprop through the model
- Updated weights

Hyperparameters

- learning rate
- LRS - learning rate scheduler
- Optimizer hyperparameters

```
for each in range(num_epochs):  
    for batch in train_dataloader:  
        outputs = model(**batch)  
        loss = outputs.loss  
        loss.backward()  
        optimizer.step() # update optimizer
```

Lamini LLAMA library in 3 lines of code

```
from llama import BasicModelRunner  
model = BasicModelRunner("EleutherAI/pythia-410m")  
model.load_data_from_jsonlines("lamini_docs.jsonl", input_key="question", output_key="answer")  
model.train(is_public=True)
```

[05_Training_lab_student.html](#)

[05_Training_lab_student.ipynb](#)

Evaluation

Evaluating generative models is very difficult.

- Human expert evaluation is the most reliable method.
- Evaluation requires good test data
 - Should be high quality
 - Accurate

- Generalized
- Not seen in the training data
- Elo comparisons are getting popular - A/B test or tournament between different LLMs

LLM Benchmarks

Includes a suite of different evaluation methods and averaging them all together to rank models

Developed by EleutherAI

- ARC - set of grade school science multiple-choice questions
- HelloSwag is a test of common sense
- MMLU (Massive Multitask Language Understanding) - multitask metric covering elementary math, US history, computer science law and more
- TruthfulQA measures a model's propensity to reproduce falsehoods commonly found online

Error Analysis

- understand base model behavior before fine-tuning
- Categorize errors: iterate on data to fix these problems
- Requires a trained model - can perform analysis on the base model before finetuning

Categories

- Misspelling
- Too long - verbose. Answering questions succinctly
- Repetitive - Curbing repetition with stop tokens. The examples in the dataset should not have much repetition as well

Evaluating fine-tuned models on ARC evaluation dataset for example would lead to the drop in scores because the model is expected to do better on the fine tuned tasks and not on the tasks meant for grade-school questions.

The performance on ARC can improve if the model is trained on the general tasks like Alpaca

[Finetuning_Pythia_70m_Simple.ipynb](#)

Practical approach to fine-tuning

1. Figure out your task
2. Collect data related to tasks inputs/outputs
3. Generate data if you don't have enough data
4. Fine-tune a small model (400M-1B) and check performance
5. Vary the amount of data you give the model
6. Evaluate the model performance whether it is going well or not
7. Collect more data to improve if not better
8. increase task complexity
9. Increase the model size for more complex tasks for performance

Tasks to fine-tune vs model-size

- Writing tasks are more harder and requires more tokens produced by models
- Harder tasks requires larger models - including multiple tasks

Model size and compute requirements

Training needs far more memory than inference. To train a model in 16GB can only fit 1B parameter model, for 2B need 256GB of GPU memory.

AWS Instance	GPUs	GPU Memory	Max inference size (# of params)	Max training size (# of tokens)
p3.2xlarge	1 V100	16GB	7B	1B
p3.8xlarge	4 V100	64GB	7B	1B
p3.16xlarge	8 V100	128GB	7B	1B
p3dn.24xlarge	8 V100	256GB	14B	2B
p4d.24xlarge	8 A100	320GB HBM2	18B	2.5B
p4de.24xlarge	8 A100	640GB HBM2e	32B	5B

There are some other ways to train the model with techniques called Parameter efficient fine-tuning (PEFT) in case you want to work with even larger models

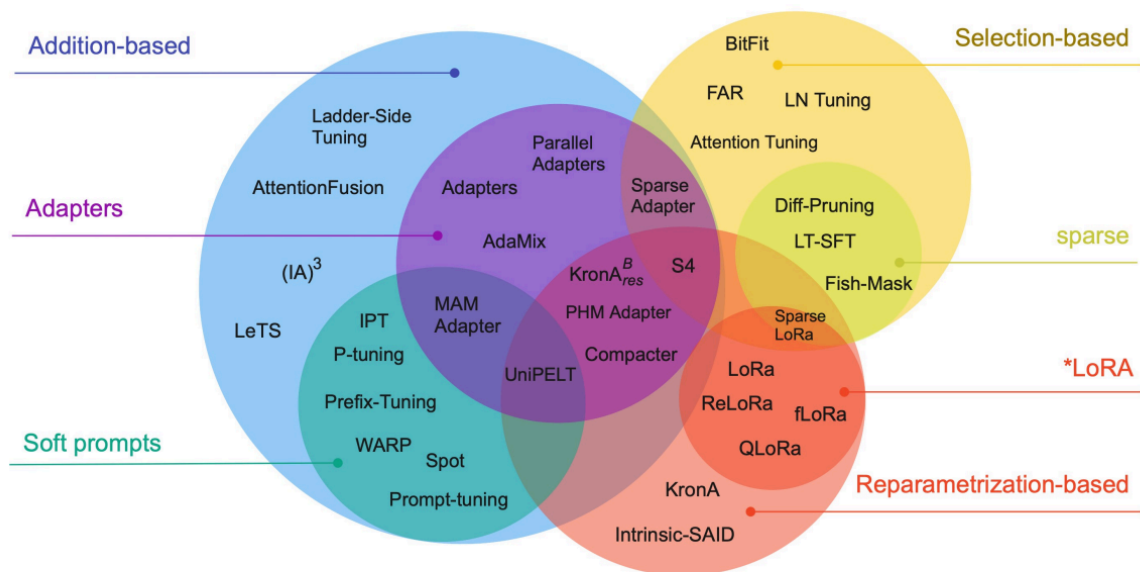


Figure 2: Parameter-efficient fine-tuning methods taxonomy. We identify three main classes of methods: **Addition-based**, **Selection-based**, and **Reparametrization-based**. Within additive methods, we distinguish two large included groups: **Adapter-like** methods and **Soft prompts**.

LoRA (Low-rank Adaptation)

- reduces the number of parameters that are required to be trained.

- For GPT-3, the number of trainable parameters reduced by 10000x - leads to less memory requirement from GPU
- Slightly below accuracy to fine-tuning
- Same inference latency
- Train new weights instead and freeze the main weights - The new weights are rank decomposition matrices of original weights' change.
 - Train separately
 - Merge with main weights at inference time